

山东大学
SHANDONG UNIVERSITY

密码分析学第四次实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：November 3, 2026

Contents

1 组员分工 **2**

2 实验环境 **2**

3 实验原理 **2**
3.1 DDT 与 LAT 2
3.1.1 DDT 2
3.1.2 LAT 3
3.2 MILP 3
3.3 点集的刻画 4
3.4 S 盒传播的建模 5
3.5 one-hot 法 6
3.6 逐点排除法 7
3.7 逻辑化简法 8
3.8 凸包法 10
3.9 全解枚举与等价性验证 14

4 实验内容 **14**
4.1 任务一：DDT 与 LAT 的建模及求解 14
4.1.1 one-hot 法 17
4.1.2 逐点排除法 19
4.1.3 逻辑化简法 20
4.1.4 凸包法 22
4.1.5 四种方法的结果对比 24
4.2 任务二：Cipherfour 算法六轮最优差分与线性路线搜索 24
4.2.1 实验步骤 24
4.2.2 实验结果及分析 28

1 组员分工

- 邱泽辉：待填写
- 江李阳：待填写
- 王健一：待填写

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS、WSL Arch Linux，Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。
- 求解与建模工具：Gurobi 及其 Python 接口 `gurobipy` 13.0.3、PyEDA 0.29.0（Espresso 逻辑化简）、SageMath 10.9（凸包构造）。
- Python 依赖管理：`uv`。

需要先安装 Gurobi, 免费的可以使用 2000 个变量和 2000 个约束条件, 在这次实验里足够了

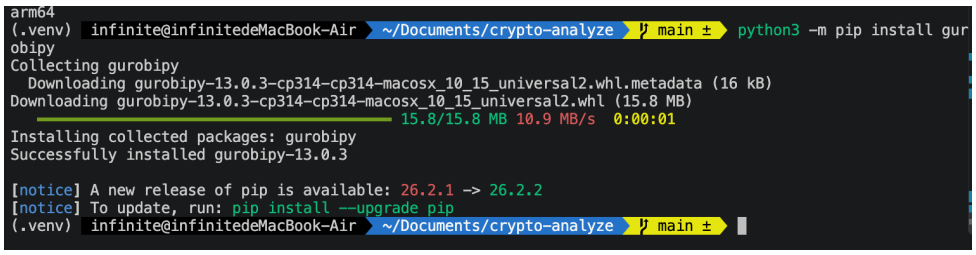


Figure 1: Gurobi 安装界面

3 实验原理

3.1 DDT 与 LAT

根据课程所学的差分分析和线性分析，这里简单复述一下 S 盒的差分分布表 (DDT) 和线性分布表 (LAT)。

3.1.1 DDT

对于一个 $m \times n$ 的 S 盒，即映射 $S : \mathbb{F}_2^m \to \mathbb{F}_2^n$，记满足输入差分 α、输出差分 β 的输入集合为

$$IN_S(\alpha, \beta) = \{X \in \mathbb{F}_2^m : S(X) \oplus S(X \oplus \alpha) = \beta\}.$$

DDT 中的表项为该集合的元素个数，即

$$\mathrm{DDT}[\alpha][\beta] = \#IN_S(\alpha, \beta).$$

3.1.2 LAT

对于同一个 $m \times n$ 的 S 盒，LAT 是一个 $2^m \times 2^n$ 的表，其中第 α 行第 β 列的表项为

$$\mathrm{LAT}[\alpha][\beta] = \#\{x \in \mathbb{F}_2^m \mid \alpha \cdot x \oplus \beta \cdot S(x) = 0\} - 2^{m-1}.$$

这里 $\cdot$ 表示模 2 内积。该表项是输入、输出掩码为 α, β 时，使线性表达式 $\alpha \cdot x \oplus \beta \cdot S(x)$ 等于 0 的输入个数与 2^{m-1} 的差，其值与输入总数 2^m 的比值恰好为线性近似的偏差：

$$\epsilon(\alpha, \beta) = \Pr[\alpha \cdot x = \beta \cdot S(x)] - \frac{1}{2} = \frac{\mathrm{LAT}[\alpha][\beta]}{2^m}.$$

对于有限的点集, 我们可以列出所有的点，那也就是说，它的边界肯定是直线

3.2 MILP

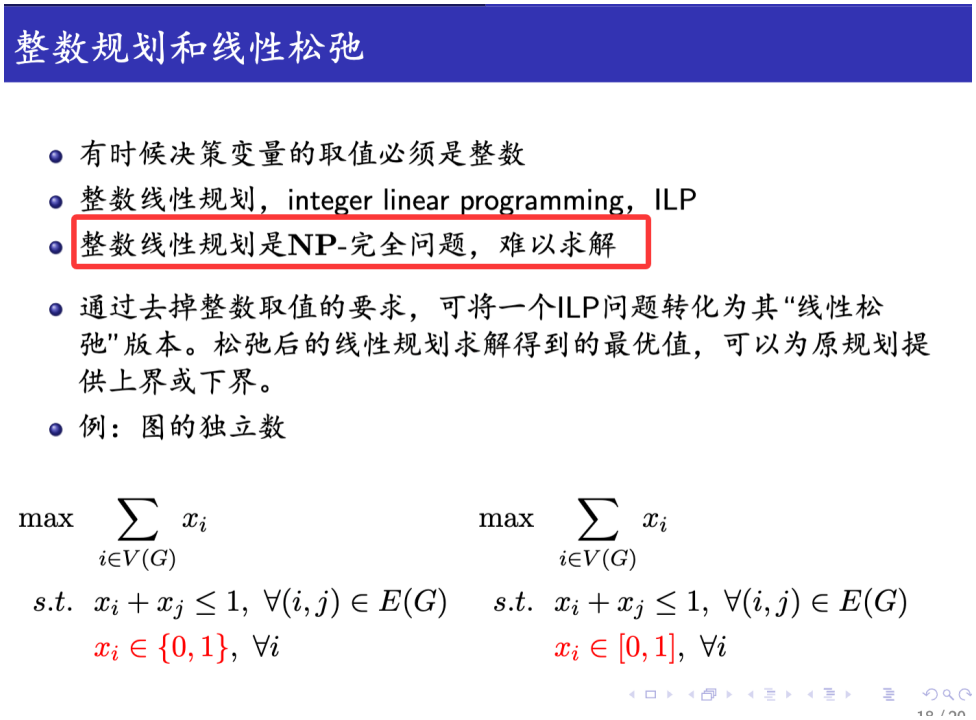
搜索差分路线是一件比较困难的事情, 在算法功底扎实的情况下，我们确实可以通过 dfs 或者动态规划来求解问题, 不过在纯人工的角度下来说，这个还是比较困难的，毕竟不同的问题我们就需要设计不同的算法, 虽然说有 AI 可以解决，但是本质上还是特题特解

所以说如果我们有一个自动化的工具，那无疑是更好的，自动化工具的本质其实就是把编程问题抽象成数学问题. 因为本质上就是优化问题，那只要我们能把问题数学建模，数学上的优化方法要远比我们编程的算法要普适得多

在我们得到了 DDT 和 LAT 之后, 我们希望把他里面的东西变成数学对象, 以致于我们可以用线性规划之类的数学优化方法来求解, 在这里我们可以用 MILP(Mixed Integer Linear Programming) 来建模, 也就是混合整数线性规划, 形式如下

$$\begin{aligned} \min \quad & c^T x + d^T y \\ \text{s.t.} \quad & Ax + By \le b \\ & x \in \mathbb{R}^n \\ & y \in \mathbb{Z}^m \end{aligned}$$

目标函数（Objective Function）：$c^T x + d^T y$，要求最大化或最小化。
约束条件（Constraints）：$Ax + By \le b$，必须是线性的，也可以包含线性等式
连续变量：$x \in \mathbb{R}^n$。
整数变量：$y \in \mathbb{Z}^m$，其中二进制变量 $y_i \in \{0, 1\}$ 是常见的特殊情况
MILP 的核心就是：目标函数和约束都是线性的，但部分或全部变量必须取整数
不过在密码学中连续的变量其实很少出现，其实这里是 ILP, 也就是 Integer Linear Programming
那这个问题我们其实在组合学图论这门课程中遇到过



那你会发现这是一个 NPC 问题, 是没有多项式时间算法的 (至少现在)

那我们规约成 NPC 问题真的合适吗? 关键是 S 盒的路线本来就是指数级别的, 那即使你用 dfs 或者动态规划，他其实也是指数复杂度的

所以说既然如此我们不如用 MILP 来建模, 因为 MILP 有很多现成的求解器, 比如 gurobi, 它是一个非常强大的求解器, 可以在很短的时间内求解大规模的 MILP 问题

3.3 点集的刻画

目标函数肯定是明确的，就是最优的路线

关键是约束条件怎么写, 在我们得到 DDT 和 LAT 之后, 那怎么判定一条路线能不能走通呢?

DDT 就统计了所有的输入差分和输出差分的组合, 如果这个组合在 DDT 中是大于 0 的, 那就说明是可以走的通的，无非就是概率大小的事

LAT 考虑的是偏差，那有正有负，只要不等于 0，就说明有机会

所以说我们不妨记

$$M[b,a] = \frac{\mathrm{DDT}[a][b]}{16}$$

这就表示从输入差分 a 到输出差分 b 的概率, 只要这个概率大于 0，那就是可达的

那我们的差分路线本质上就是在这些可达的地方不断移动，这个所谓的 $M[b,a]$ 就很像平面直角坐标系里的一个点

那我们把这些可达的点

$$M[b,a] \neq 0$$

构成一个点集, 记作 $supp(M)$

$$V_M = \{(a,b) \mid (a,b) \in \mathrm{supp}(M)\}$$

那这个点集就是差分传播的研究对象线性近似路线也是一样的，我们用相关度来表示

$$C[b,a] = \frac{\text{LAT}[a][b]}{8}$$

那我们把这些可达的点

$$C[b,a] \neq 0$$

构成一个点集, 记作 $supp(C)$

$$V_C = \{(a,b) \mid (a,b) \in \text{supp}(C)\}$$

那你说建模很简单啊，约束关系就写 $v \in V, v \notin V$ 不就行了

对于人来说这个属不属于当然简单，不过它并不是一个线性运算啊

这个属于关系没办法写成加减乘除的形式，在编程里判断属于关系也是比普通的运算复杂的，虽然说 python 有 in 运算，不过那也只是封装了而已

那你像 C 语言中我们判断属于关系，朴素的情况下就是遍历数组的所有元素，判断是否相等

所以说这个属于关系和线性是搭不上边的

进一步说我们考虑一个例子

$$(a,b) \in V$$

例如：

$$S = \{(1,2),(1,3),(2,1)\}$$

约束实际上等价于

$$(a=1 \wedge b=2) \vee (a=1 \wedge b=3) \vee (a=2 \wedge b=1)$$

那属于的本质其实可以理解为析取的运算，但是这个显然不是线性的

那所以说关键就是把点集建模成数学变量，让它能用线性的式子去刻画属于关系

3.4 S 盒传播的建模

我们可以随便建模，但是怎么验证我们约束的正确性呢？

一个比较耍赖的逻辑是，求解器求出来的解，肯定是满足约束的

试想一下，如果我们直接把目标函数设成常数会得到什么？

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & \ldots \end{aligned} \tag{1}$$

如果目标函数是常数，那么所有满足约束条件的解都是最优解

也就是说我们的第一步是，找到一组约束和变量，使得他们的解是点集 V, 那这就能说明我们建模是正确的

在验证单个 S 盒的传播约束后，要搜索整条差分或线性路线，还需按轮复制这些约束，加入轮间置换连接关系与局部概率或相关度权重，再设置相应的优化目标。

我们先把坐标写成二进制的形式

Cipherfour 使用 4×4 的 S 盒。对于 (a,b)，按大端序展开

$$(a,b) \to (a_3,a_2,a_1,a_0,b_3,b_2,b_1,b_0) \in \{0,1\}^8,$$

这个向量我们记作 $\text{bits}(a,b)$ 那你说本来就两个变量，为什么要弄成 8 个变量呢，额这其实另有考量，我们后面再谈

3.5 one-hot 法

基本原理 一个最直接的想法就是，我们直接就给所有点一个变量，集合的属于其实就是 0,1 关系嘛，0 就是不属于，1 就是属于

所以说我们给每个点一个 0,1 变量, 记作 $\lambda_{(a,b)}$，如果点 (a,b) 属于点集, 那么 $\lambda_{(a,b)}=1$，否则 $\lambda_{(a,b)}=0$

那你求点集里的点，其实就是 $\lambda_{(a,b)}=1$, 然后其他都是 0

所以约束条件就出来了，

$$\sum_{(a,b)\in V}\lambda_{(a,b)}=1,$$

数学建模

$$\begin{aligned}\min\quad & 0\\ \text{s.t.}\quad & \sum_{(a,b)\in V}\lambda_{(a,b)}=1\\ & x=\sum_{(a,b)\in V}\lambda_{(a,b)}\,\mathrm{bits}(a,b)\\ & \lambda_{(a,b)}\in\{0,1\},\forall(a,b)\in V\end{aligned}\tag{2}$$

正确性分析 对于模型的任意可行解，由 $\lambda_{(a,b)}\in\{0,1\}$ 以及

$$\sum_{(a,b)\in V}\lambda_{(a,b)}=1$$

可知，存在唯一的 $(a^*,b^*)\in V$，使得 $\lambda_{(a^*,b^*)}=1$，其余选择变量均为 0

将其代入坐标约束

$$x_j=\sum_{(a,b)\in V}\mathrm{bits}(a,b)_j\lambda_{(a,b)}$$

可得

$$x_j=\mathrm{bits}(a^*,b^*)_j,\quad j=0,1,\ldots,7.$$

因此，$x=\mathrm{bits}(a^*,b^*)$，即 x 对应于 V 中的点，不会有不可达的点

反过来，任取 $(a,b)\in V$，令

$$x=\mathrm{bits}(a,b),\qquad \lambda_{(a,b)}=1,$$

其余选择变量均为 0，则显然满足模型中的全部约束

用于差分与线性传播 差分建模时取 $V=V_{\mathrm{D}}$，每个选择变量对应一个 DDT 非零位置；线性建模时取 $V=V_{\mathrm{L}}$，每个选择变量对应一个 LAT 非零位置。也可以直接用 (a,b) 给选择变量编号，记作 $\lambda_{(a,b)}$，从而与原表中的位置一一对应。两类模型分别建立各自的选择变量，坐标约束形式保持一致；全零点的选择变量也需要保留，以表示未活跃 S 盒。表项的具体数值不作为这里的优化目标，枚举结果应覆盖全部可达点。

3.6 逐点排除法

基本原理 简单来说就是正难则反，我们如果能求出点集的补集，那也能刻画出点集

也就是说我们排除掉不可达的点，剩下的就是可达的点了，那怎么排除呢？

我们知道两个点重合就意味他们的距离为 0，如果距离大于 0，那就意味着两个点不是同一个点，可以用这个特性来排除

那距离首先想到的就是 Euclid 距离

设不可达点集为

$$Q = \{0,1\}^8 \setminus V.$$

我们对于任意 $q = (q_1, \ldots, q_8) \in Q$

$$d(x,q) = \sqrt{\sum_{i=1}^{8}(x_i - q_i)^2}$$

我们知道二进制不管怎么加减无非就是 0,1,-1, 那么也就是说 $(x_i - q_i)^2 = |x_i - q_i|$

那我们再按 q_i 的值来分的话

如果 $q_i = 0$, 那 $|x_i - q_i| = |x_i| = x_i$,

如果 $q_i = 1$, 那 $|x_i - q_i| = |x_i - 1| = 1 - x_i$, 因为 $x_i - 1 \le 0$

那么也就是说

$$d(x,q) = \sqrt{\sum_{j:q_j=0} x_j + \sum_{j:q_j=1}(1 - x_j)}$$

那我们希望是整数，干脆就不要开根号了，反正没什么区别

那这其实就是 Hamming 距离

对于任意 $q = (q_1, \ldots, q_8) \in Q$，定义 x 与 q 的 Hamming 距离

$$d(x,q) = \sum_{j:q_j=0} x_j + \sum_{j:q_j=1}(1 - x_j).$$

数学建模

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & \sum_{j:q_j=0} x_j + \sum_{j:q_j=1}(1 - x_j) \ge 1, \quad q \in Q, \\ & x_j \in \{0,1\}, \qquad\qquad j = 1, \ldots, 8. \end{aligned}$$

也可以进行移项

$$\sum_{j:q_j=0} x_j - \sum_{j:q_j=1} x_j \ge 1 - \sum_{j=1}^{8} q_j, \qquad q \in Q$$

例如，输入为 0、输出为 1 对应的点为 $q = 0000\,0001$, 我们如果要排除它，就可以写成

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + (1 - x_8) \ge 1.$$

正确性分析 对于任意不可达点 $q \in Q$，若 $x = q$，则

$$d(x,q) = \sum_{j:q_j=0} x_j + \sum_{j:q_j=1} (1 - x_j) = 0,$$

不满足约束 $d(x,q) \geq 1$，所以不可达点都被排除了

反之，若 $x \neq q$，由于 $x, q \in \{0,1\}^8$，则至少存在一个坐标 j 使得 $x_j \neq q_j$，因此 $d(x,q) \geq 1$，也就是说只排除 q 这一个点

对于任意可达点 $x \in V$，由于 V 与 Q 不相交，因此对于所有 $q \in Q$，均有 $x \neq q$，从而满足全部约束，不会有可达点遗漏

反过来，若 $x \notin V$，则根据 $Q = \{0,1\}^8 \setminus V$，必有 $x \in Q$。此时模型中存在针对 $q = x$ 的约束，而 $d(x,q) = 0$，与 $d(x,q) \geq 1$ 矛盾, 所以不会引入任何不可达点。

综上，模型的二进制可行解集合恰好为 V，即

$$\left\{ x \in \{0,1\}^8 \;\middle|\; \sum_{j:q_j=0} x_j + \sum_{j:q_j=1} (1 - x_j) \geq 1,\ \forall q \in Q \right\} = V.$$

用于差分与线性传播 差分建模时，将 DDT 零位置对应的八位向量作为 Q；线性建模时，将 LAT 零位置对应的八位向量作为 Q。两种情况下均只使用原始变量 x，不需要额外的选择变量。

3.7 逻辑化简法

基本原理 前面我们通过 Hamming 距离，为每个不可达点构造一条线性不等式，从而将其排除。

但是这样有一个问题，就是如果不可达点太多，那么需要添加的约束也会非常多

那能不能用一条约束同时排除多个不可达点呢？

若干非法点若只在某些位上取值相同、其余位任意，就构成一个 cube；一条只看固定位的「距离 ≥ 1」把整个 cube 一次排除。例：

cube	包含的非法点				不等式
000|1−−	000|100	000|101	000|110	000|111	$a_2 + a_1 + a_0 + (1 - b_2) \geq 1$

Espresso 读入非法点的真值表，输出一组覆盖全部非法点、不含合法点的 cube，并尽量少；每个 cube 一条不等式。

χ_3：35 个非法点 → 14 个 cube，14 条；

> 逐点排除合并成逐块排除：χ_3 上 35 → 14；每条不等式只看 cube 里固定的位。

就比如课件里这个例子, 不可达点的前四位都是 0001, 那这样我们就可以合并成一个不等式，本来是需要四个的

逻辑化简法将这些点作为布尔函数的取值为 1 的输入，其实就有点像数字逻辑里的卡诺图化简

通过 Espresso 等工具合并为 cube，再对每个 cube 建立一条排除约束

一条这样的约束可以同时排除该 cube 覆盖的全部不可达的点

不可达点的布尔表示 首先，设不可达点集合为

$$Q = \{0,1\}^8 \setminus V.$$

我们定义一个布尔函数

$$f_{\text{bad}}(x) = \begin{cases} 1, & x \in Q, \\ 0, & x \in V. \end{cases}$$

接下来，我们可以把这个函数的真值表交给逻辑化简工具，尝试把多个不可达点合并起来

一般来说，一个 cube 可以写成

$$c = (c_1, \ldots, c_8) \in \{0, 1, -\}^8.$$

其中 0 和 1 表示对应位置必须取固定值，而 $-$ 表示这一位可以任意取值

为了方便后面构造约束，我们记

$$I_0(c) = \{j \mid c_j = 0\}, \qquad I_1(c) = \{j \mid c_j = 1\}.$$

那么，一个 cube 所表示的全部二进制点就是

$$B(c) = \left\{ x \in \{0,1\}^8 \;\middle|\; x_j = 0 \ (j \in I_0(c)),\ x_j = 1 \ (j \in I_1(c)) \right\}.$$

假设经过逻辑化简后，我们得到了一个 cube 集合 $\mathcal{C}$，那么我们希望这些 cube 恰好覆盖所有不可达点，即

$$\bigcup_{c \in \mathcal{C}} B(c) = Q. \tag{3}$$

这里要注意，所谓恰好覆盖，不仅要求所有不可达点都包含在这些 cube 中，还要求不能把任何可达点也包含进去。

讲义使用 `espresso-logic` 读取 PLA 格式的非法点列表：本实验应声明八个输入、一个输出，输入标签按 $a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0$ 排列，每个非法点的输出标为 1，未列出的点按对应 PLA 输入模式解释为输出 0，不能当作任意取值。读取化简结果时，逐行提取输出为 1 的 cube；其中的`-`表示该位不限，`.p` 字段表示输出的 cube 数量。无论采用讲义中的`-Dexact` 方式，还是通过其他接口调用 Espresso，后续线性化都依赖式(3)所要求的精确覆盖关系。

cube 的线性约束 如果要排除一个 cube，只需要保证 x 至少有一个固定坐标与 cube 不一致即可

至于那些取值为 $-$ 的坐标，就不需要考虑了

对于一个 cube c，我们可以构造如下约束：

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & \sum_{j \in I_0(c)} x_j + \sum_{j \in I_1(c)} (1 - x_j) \geq 1, \quad \forall c \in \mathcal{C}, \\ & x_j \in \{0,1\}, \qquad\qquad\qquad\qquad j = 1, \ldots, 8. \end{aligned} \tag{4}$$

我们也可以将常数项移到右边，得到

$$\sum_{j \in I_0(c)} x_j - \sum_{j \in I_1(c)} x_j \geq 1 - |I_1(c)|.$$

例如，假设有一个 cube

$$c = 0000\,000 - .$$

那么我们只需要构造

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \geq 1.$$

这样就同时排除了 0000 0000 和 0000 0001 两个点

正确性分析 对于任意一个 cube $c \in \mathcal{C}$，如果 $x \in B(c)$，那么 x 的所有固定坐标都与 c 一致。

也就是说，当 $c_j = 0$ 时，有 $x_j = 0$；当 $c_j = 1$ 时，有 $x_j = 1$。

因此，

$$\sum_{j \in I_0(c)} x_j + \sum_{j \in I_1(c)} (1 - x_j) = 0,$$

不满足约束条件，所以 x 被排除。

反过来，如果 $x \notin B(c)$，那么至少存在一个固定坐标与 c 不同，因此上式左边至少为 1，满足对应的约束。

这就说明，一条约束恰好可以排除一个 cube 覆盖的全部二进制点，而不会影响 cube 之外的点。

接下来考虑所有 cube 构成的集合 $\mathcal{C}$。

根据前面逻辑化简得到的覆盖关系

$$\bigcup_{c \in \mathcal{C}} B(c) = Q,$$

我们知道，所有 cube 恰好覆盖全部不可达点。

因此，对于任意可达点 $x \in V$，它不属于任何 cube 覆盖的点集，所以能够满足全部约束，模型不会遗漏合法传播。

反过来，如果 $x \notin V$，那么 $x \in Q$，根据覆盖关系，必然存在某个 cube $c \in \mathcal{C}$ 使得 $x \in B(c)$，从而违反对应的约束。

因此，所有不可达点都会被排除。

综上，模型的二进制可行解集合恰好为 V，即

$$\left\{ x \in \{0,1\}^8 \;\middle|\; \sum_{j \in I_0(c)} x_j + \sum_{j \in I_1(c)} (1 - x_j) \geq 1,\ \forall c \in \mathcal{C} \right\} = V.$$

这样，我们就利用逻辑化简，将原来针对每个不可达点的约束，合并成了针对每个 cube 的约束。

对于差分传播，我们可以对 DDT 中值为 0 的位置进行化简；对于线性传播，则可以对 LAT 中值为 0 的位置进行化简。

3.8 凸包法

基本原理 首先先介绍一下凸包

凸集，就是集合中任意两点之间的线段都完全属于这个集合：

$$\forall x, y \in C,\ \forall \lambda \in [0,1], \quad \lambda x + (1 - \lambda) y \in C$$

对有限集合 P，它的凸包 conv(P) 是 $\mathbb{R}^n$ 中包含 P 的最小凸集

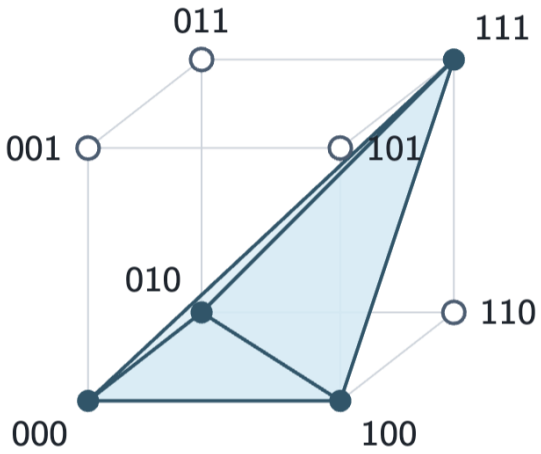
那么也就是说，凸包其实就是把集合里的点都连起来，然后把扫过的所有区域都包含进来

那前面的 one-hot 方法有一个问题就是变量太多了，每个点都要一个

那就和前面逻辑化简类似，我们尽可能把能合并的合并

那凸包就是包含一个点集的最小凸集。对于有限个点来说，凸包就可以把这个集合全部囊括

凸包还包含点之间的连续区域；只有再要求 $x \in \{0,1\}^8$，才能用后文的性质保证可行点仍属于原点集



从直觉来看，我们把边缘的点连成凸多边形就能变成凸包，因为把所有点都包住了，也就是说边界都是像图中的直线

那我们毕竟也不是数学系，这个就不证明了

凸多边形在高维叫多胞体，可以通过有限个线性不等式和等式来描述，这种表示方法就称为 H 表示

可以写成

$$P = \{x \in \mathbb{R}^8 \mid Ax + b \geq 0,\ Cx + d = 0\}.$$

其中 A 和 b 分别表示不等式的系数矩阵和常数向量，C 和 d 则表示等式的系数矩阵和常数向量。

因此，我们只需要将这些系数整理成矩阵形式，就可以得到完整的凸包模型：

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & Ax + b \geq 0, \\ & Cx + d = 0, \\ & x \in \{0,1\}^8. \end{aligned} \tag{5}$$

正确性分析 首先，对于任意可达点 $x \in V$，根据凸包的定义，显然有 $x \in P$, 所以不会漏点

不过这里有一个问题：凸包中除了原来的可达点，还会包含这些点之间的连续区域，那会不会引入新的点呢

假设存在 $x \in P$，$V = \{v^{(1)}, v^{(2)}, ...\}$
那么根据凸包的定义，存在一组非负实数 θ_i，使得

$$x = \sum_{i=1}^{N} \theta_i v^{(i)}, \qquad \theta_i \geq 0, \quad \sum_{i=1}^{N} \theta_i = 1.$$

由于 x 和 $v^{(i)}$ 的每个坐标都只能取 0 或 1
如果 $x_j = 0$，那么

$$0 = \sum_{i=1}^{N} \theta_i v_j^{(i)}.$$

由于每一项都非负，所以对所有有 $\theta_i > 0$ 的项，必有 $v_j^{(i)} = 0$
同理，如果 $x_j = 1$，那么

$$1 = \sum_{i=1}^{N} \theta_i v_j^{(i)}.$$

因为所有 $v_j^{(i)} \leq 1$，而 θ_i 非负且和为 1，所以对所有 $\theta_i > 0$ 的项，必有 $v_j^{(i)} = 1$
因此任取一个 $\theta_i > 0$ 的项，均有 $x = v^{(i)}$
因此，x 本身必然就是 V 中的某个点，不会产生新的点
综上，

$$P \cap \{0, 1\}^8 = V.$$

也就是说 $\text{conv}(V) \cap \{0, 1\}^8 = V$
正是这个性质才让我们把 (a, b) 编码成 bit, 因为你如果是单纯的整数的话，就没有这么好的性质了

基于不可达点覆盖的贪心筛选 那我们的 H 表示可能依旧得到很多不等式，我们也需要合并化简
这里我们可以采用贪心算法
假设凸包的 H 表示为

$$Ax + b \geq 0, \qquad Cx + d = 0.$$

对于其中第 i 条不等式，我们记为

$$A_i x + b_i \geq 0,$$

如果某个不可达点 $q \in Q$ 满足

$$A_i q + b_i < 0,$$

因此，我们可以把第 i 条不等式能够排除的全部不可达点记为

$$R_i = \{q \in Q \mid A_i q + b_i < 0\}.$$

这样，每条不等式都对应一个不可达点集合
而我们希望从这些集合中选出尽可能少的几个，使它们能够覆盖全部不可达点，那就只需要这么几个不等式就够了
这实际上就是一个集合覆盖问题。

不过需要注意，H 表示中除了不等式，还可能包含等式

由于等式约束需要保留，因此那些已经违反等式的不可达点，就没有必要再通过不等式排除

我们可以先将满足

$$Cq + d \neq 0$$

的不可达点从 Q 中删除。

剩下的不可达点记为

$$Q_{\text{remaining}}.$$

接下来，我们每次从尚未选中的不等式中，找出能够排除最多剩余不可达点的那一条。

具体来说，选择

$$i^* = \arg\max_{i \in \text{pool}} |R_i \cap Q_{\text{remaining}}|.$$

其中 pool 表示尚未选中的候选不等式集合。

选中后，将这条不等式加入保留集合，并更新剩余点集：

$$Q_{\text{remaining}} \leftarrow Q_{\text{remaining}} \setminus R_{i^*}.$$

然后继续寻找下一条能够排除最多剩余不可达点的不等式。

不断重复这个过程，直到

$$Q_{\text{remaining}} = \varnothing.$$

这就说明，所有不可达点都已经被选中的约束排除了

在具体代码实现中，我们可以使用三个列表：

- `remaining`：保存尚未排除的不可达点；
- `pool`：保存尚未选中的候选不等式；
- `chosen`：保存已经选中的不等式。

每次从 `pool` 中选择能够排除最多剩余点的不等式，将其加入 `chosen`，同时从 `remaining` 中删除被它排除的点。

约简模型的正确性分析 假设经过贪心筛选后，我们得到了保留下来的不等式集合

将这些不等式对应的系数重新组成矩阵 A' 和向量 b'，那么约简后的模型可以写成

$$\begin{aligned} \min \quad & 0 \\ \text{s.t.} \quad & A'x + b' \geq 0, \\ & Cx + d = 0, \\ & x \in \{0,1\}^8. \end{aligned} \tag{6}$$

接下来，我们证明约简后的模型仍然成立

首先，所有保留下来的约束都来自原来的完整 H 表示

由于所有可达点都满足完整 H 表示，那么删除其中一部分不等式之后，可达点自然仍然满足剩下的约束

反过来，对于任意不可达点 $q \in Q$，我们分两种情况讨论

如果 q 不满足等式约束

$$Cq + d = 0,$$

那么它已经被等式排除，不可能成为模型的可行解

如果 q 满足全部等式约束，那么它就属于贪心算法初始时尚未排除的点集

由于贪心算法只有在所有剩余不可达点都被排除之后才会结束，因此必然存在某条被保留的不等式，使得

$$A_i'q + b_i' < 0.$$

所以 q 同样不满足约简后的模型。

这说明全部不可达点都被排除

当然贪心算法求出来的不一定是最优解，不过我们也实现了减少约束数量的目的

最后，对于差分传播和线性传播，我们分别利用 DDT 和 LAT 中的非零位置构造可达点集，再独立计算凸包并进行约束筛选

3.9 全解枚举与等价性验证

模型建立后将目标函数设为常数 0，仅枚举满足约束的全部二进制解，不按概率或相关度大小筛选。Gurobi 的 `PoolSearchMode=2` 用于系统搜索多个解，`PoolSolutions` 指定希望收集的解数。原始变量只有八位，取值空间为 $\{0,1\}^8$；

对于上述四种模型，one-hot 的选择变量由可达点唯一确定，其余模型不含辅助变量，因此模型的不同完整解也不超过原始八位点的取值总数。将解池容量设为严格大于这一上界，并在求解以 `OPTIMAL` 状态结束且解池未满时，才确认全部解已收集。

读取解池时，逐个设置 `SolutionNumber`，再获取该解中各原始变量的值。

讲义使用 `Xn` 属性，新版本 Gurobi 也提供 `PoolNX` 属性。对 one-hot 模型，还应核对恰有一个选择变量取 1，且该选择变量对应的点与原始八位变量相同。

记枚举结果在原始变量上的投影集合为 S_{enum}，则分别计算

$$V \setminus S_{\text{enum}}, \qquad S_{\text{enum}} \setminus V.$$

前者表示漏掉的合法点，后者表示多出的非法点。只有枚举已经完成且两个差集均为空，才说明模型的二进制解恰好为 V。差分与线性两类点集需要分别进行验证；解数相同只是必要条件，还需逐点比较。实际约束系数、条数、枚举结果及求解状态在实验内容部分记录。

4 实验内容

4.1 任务一：DDT 与 LAT 的建模及求解

本任务针对 CipherFour 的 4 比特 S 盒，分别采用 one-hot 法、逐点排除法、逻辑化简法和凸包法建立混合整数线性规划模型，并枚举各模型的全部二进制解。首先直接计算 S 盒的 DDT 和 LAT，再以表中的非零位置为参照，检查四种模型得到的输入输出对是否一致。DDT 中的 (a, b) 表示输入差分和输出差分，LAT 中的 (a, b) 表示输入掩码和输出掩码。

公共 DDT、LAT 的计算 四种方法共用 `ddt_lat.py` 中的表格计算函数。对于 DDT，遍历输入差分 a 和输入 x，计算 $S(x) \oplus S(x \oplus a)$，并在对应输出差分的位置累计计数。关键代码如下：

```
for a in range(SIZE):
    for x in range(SIZE):
        b = SBOX[x] ^ SBOX[x ^ a]
        table[a][b] += 1
```

对于 LAT，遍历输入、输出掩码 a, b，统计 $a \cdot x = b \cdot S(x)$ 成立的输入个数，再减去 8。程序通过按位与和 `bit_count()` 计算模 2 内积，其核心代码为：

```
equal_count = 0
for x in range(SIZE):
    input_parity = (a & x).bit_count() & 1
    output_parity = (b & SBOX[x]).bit_count() & 1
    if input_parity == output_parity:
        equal_count += 1
table[a][b] = equal_count - SIZE // 2
```

运行遍历计算得到图 2中的两张表。行下标表示输入，列下标表示输出，均使用十六进制。DDT 共有 72 个非零项，LAT 共有 122 个非零项。因此，差分模型应保留 72 个合法点，线性模型应保留 122 个合法点。

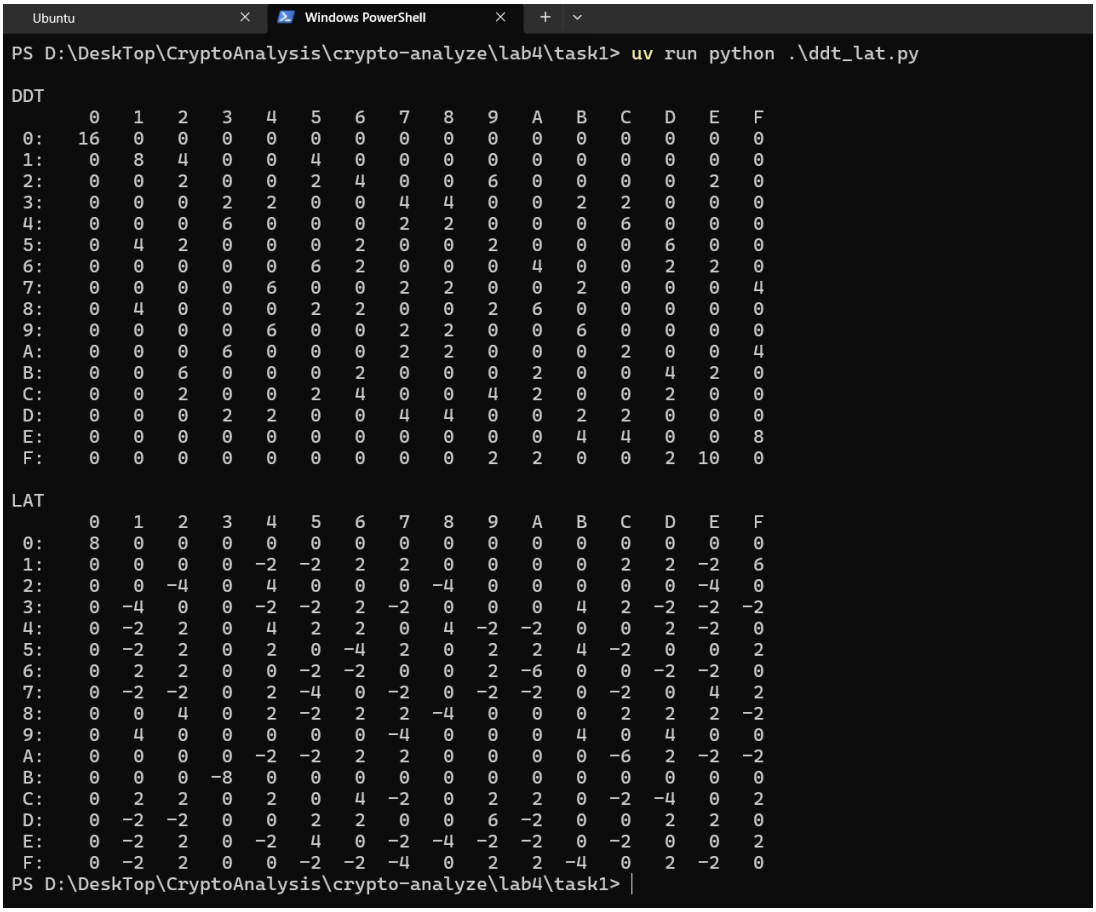


Figure 2: 直接遍历计算得到的 S 盒 DDT 与 LAT

统一的编码与枚举方式 程序将每个输入输出用大端序展开

$$x = (a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0),$$

为枚举所有合法点，各模型的目标函数均设为常数 0，并启用系统性解池搜索：

```
model.setObjective(0, gp.GRB.MINIMIZE)
model.Params.PoolSearchMode = 2
capacity = 2**8 + 1
model.Params.PoolSolutions = capacity
model.Params.MIPGap = 0
model.Params.MIPGapAbs = 0
model.optimize()
```

八位坐标共有 $2^8 = 256$ 种取值，合法点与完整变量赋值一一对应，因此将解池容量设为 257。程序在确认求解达到最优且解池未满后，逐个读取解池中的解，按统一位序还原输入输出，并以十六进制排序打印。每个解表示表中的一个非零位置，其具体计数或偏差仍由图 2给出。

4.1.1 one-hot 法

实验过程与关键代码 one-hot 法从 DDT 或 LAT 中提取所有非零位置，组成可达点列表 `points`。随后对每个可达输入输出对设置一个二进制选择变量 $\lambda_{a,b}$，并要求所有选择变量之和为 1。这样，每次求解都恰好选中一个可达点。

在 `one_hot.py` 中，通过 `bits(a,b)` 计算可达点的八位编码，并建立 8 个原始变量和相应的选择变量：

```
V = [(a, b) for a in range(SIZE) for b in range(SIZE) if table[a][b] != 0]
x = model.addVars(8, vtype=gp.GRB.BINARY)
lam = model.addVars(V, vtype=gp.GRB.BINARY)
model.addConstr(lam.sum() == 1)
```

选择变量确定可达点后，还需要使 x 的每一位等于该点的对应坐标。对于第 j 位，只累加该位为 1 的可达点所对应的选择变量：

```
for j in range(8):
    model.addConstr(x[j] == gp.quicksum(lam[p] for p in V if bits(*p)[j] ==
    1))
```

因此，模型包含 1 条选点等式和 8 条坐标恢复等式，共 9 条等式。由于可达点列表中的每个输入输出对互不相同，一组八位坐标也唯一确定被选中的 λ。建立约束后，使用前面的解池参数分别枚举 DDT 模型与 LAT 模型。

实验结果与分析 代码运行结果如图 3所示。



Figure 3: one-hot 法的 DDT、LAT 模型全解枚举结果

DDT 模型有 72 个选择变量，加上 8 个原始变量，共 $72 + 8 = 80$ 个二进制变量；

LAT 模型对应 $122 + 8 = 130$ 个二进制变量。两者均有 9 条等式，与上述建模过程一致。求解得到的 DDT 解数为 72，LAT 解数为 122，按输入分组后与图 2中的非零项一致，所以说，one-hot 法的建模与求解结果正确

图 4是开启 Gurobi 过程日志时的一次运行；当前脚本默认关闭过程日志，只打印最终枚举结果。

```
(.venv) infinite@infinitedeMacBook-Air ~/Documents/crypto-analyze  main ± /Users/in
finite/Documents/crypto-analyze/.venv/bin/python /Users/infinite/Documents/crypto-analyze/l
ab4/task1/one_hot/one_hot.py

Optimize a model with 9 rows, 80 columns and 382 nonzeros (Min)
Model fingerprint: 0x14d77795
Model has 0 linear objective coefficients
Variable types: 0 continuous, 80 integer (80 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [0e+00, 0e+00]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 1e+00]

Found heuristic solution: objective 0.0000000
Presolve removed 8 rows and 8 columns
Presolve time: 0.00s
Presolved: 1 rows, 72 columns, 72 nonzeros
Variable types: 0 continuous, 72 integer (72 binary)

Root relaxation: objective 0.000000e+00, 1 iterations, 0.00 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0          -    0         0.00000    0.00000  0.00%     -    0s

Optimal solution found at node 0 - now completing solution pool...

    Nodes    |    Current Node    |      Pool Obj. Bounds     |     Work
             |                    |   Worst                   |
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0          -    0               -    0.00000      -     -    0s
     0     0          -    0               -    0.00000      -     -    0s
     0     0          -    0               -    0.00000      -     -    0s
     0     2          -    0               -    0.00000      -     -    0s

Explored 143 nodes (3 simplex iterations) in 0.00 seconds (0.00 work units)
Thread count was 10 (of 10 available processors)

Solution count 72: 0 0 0 ... 0
No other solutions better than 0

Optimal solution found (tolerance 0.00e+00)
Best objective 0.000000000000e+00, best bound 0.000000000000e+00, gap 0.0000%

DDT one-hot: 变量数=80, 等式数=9, 解数=72
全部解（输入，输出；十六进制）：
(0,0) (1,1) (1,2) (1,5) (2,2) (2,5) (2,6) (2,9) (2,E) (3,3) (3,4) (3,7) (3,8) (3,B) (3,C) (
4,3) (4,7) (4,8) (4,C) (5,1) (5,2) (5,6) (5,9) (5,D) (6,5) (6,6) (6,A) (6,D) (6,E) (7,4) (7
,7) (7,8) (7,B) (7,F) (8,1) (8,5) (8,6) (8,9) (8,A) (9,4) (9,7) (9,8) (9,B) (A,3) (A,7) (A,
8) (A,C) (A,F) (B,2) (B,6) (B,A) (B,D) (B,E) (C,2) (C,5) (C,6) (C,9) (C,A) (C,D) (D,3) (D,4
) (D,7) (D,8) (D,B) (D,C) (E,B) (E,C) (E,F) (F,9) (F,A) (F,D) (F,E)
```

Figure 4: 开启过程日志时的 one-hot 模型预求解信息

我们发现 gurobi 会进行一个 presolve 的流程，这会使得变量缩减到 72 个, 因为剩下那 8 个变量其实就是我们的 bits(a,b)

那这些是可以由 λ 计算出来的，所以就不需要保留，不过这个属于求解器内部的事了，我们还是按 80 个变量来算

4.1.2 逐点排除法

实验过程与关键代码 逐点排除法先找出表中所有取值为 0 的位置，组成非法点列表 `forbidden`。模型只设置 8 个原始二进制变量，然后对每个非法点添加一条排除不等式。由于 LAT 中正值和负值都属于合法表项，这里统一使用 `table[a][b] == 0` 提取非法点。

对于非法点 q，当 $q_j = 0$ 时，x_j 表示当前点在第 j 位是否与它不同；当 $q_j = 1$ 时，则由 $1 - x_j$ 表示。因此，将这 8 项相加并要求结果至少为 1，就能保证当前点至少有一位不同于 q。实现代码如下：

```
forbidden = [(a, b) for a in range(SIZE) for b in range(SIZE)
             if table[a][b] == 0]
x = model.addVars(8, vtype=gp.GRB.BINARY)

for point in forbidden:
    q = bits(*point)
    model.addConstr(gp.quicksum(
        x[j] if q[j] == 0 else 1 - x[j]
        for j in range(8)
    ) >= 1)
```

每条不等式只排除对应的一个非法二进制点。添加全部约束后，剩余可行点就是表中的非零位置，再由 Gurobi 按统一参数进行枚举。

实验结果与分析 代码运行结果如图 5所示。

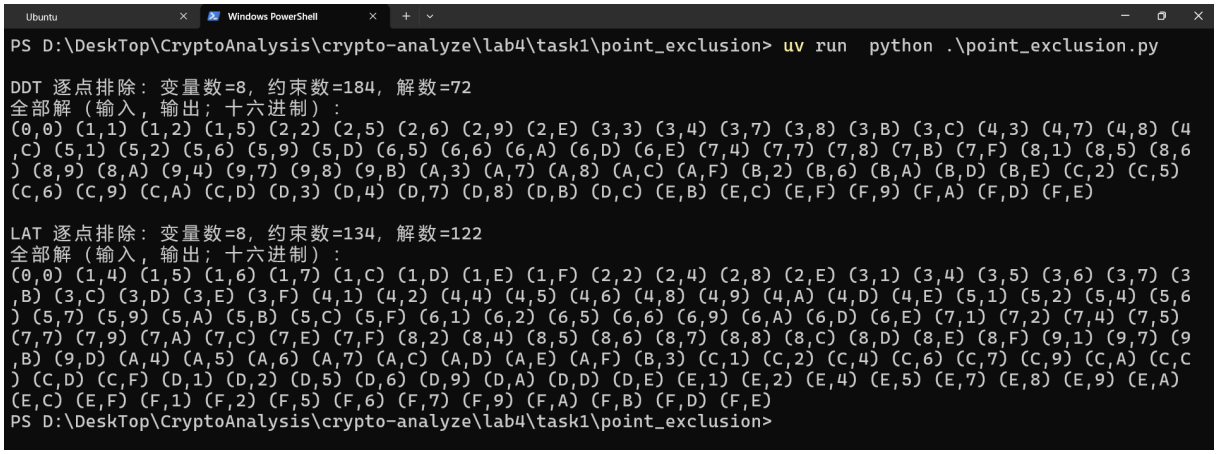


Figure 5: 逐点排除法的 DDT、LAT 模型全解枚举结果

输入、输出各有 16 种取值，因此共有 $16 \times 16 = 256$ 个候选点。DDT 中有 72 个合法点，需要排除 $256 - 72 = 184$ 个点，对应 184 条不等式；LAT 中有 122 个合法点，需要排除 $256 - 122 = 134$ 个点，对应 134 条不等式。两种模型均只有 8 个二进制变量，与程序输出一致。

枚举得到 72 个差分合法点和 122 个线性合法点，具体输入输出对与 one-hot 法及原始表的非零位置完全一致。该方法保持 8 个原始变量，通过 184 条或 134 条不等式逐点排除非法点，实现了相同的传播关系。

4.1.3 逻辑化简法

非法点函数的构造与化简 逐点排除法中，每个非法点都需要一条不等式。逻辑化简法进一步考虑将多个非法点合并处理：先构造一个八输入布尔函数，使其在非法点上取 1，在合法点上取 0，再使用 Espresso 将这个函数化简为若干乘积项。每个乘积项对应一个逻辑立方体，其中部分坐标固定为 0 或 1，其余坐标可以任意取值。

本实验在 `logic_minimization.py` 中通过 PyEDA 提供的 `espresso_exprs` 接口调用 Espresso。[1]程序按公共位序建立 8 个布尔变量，对每个零值位置生成一个最小项，再将所有最小项取或：

```
variables = exprvars("x", 8)
minterms = []
for a in range(SIZE):
    for b in range(SIZE):
        if table[a][b] == 0:
            point = bits(a, b)
            minterms.append(And(*(
                variable if value else ~variable
                for variable, value in zip(variables, point)
            )))

forbidden_function = Or(*minterms).to_dnf()
minimized, = espresso_exprs(forbidden_function)
```

化简后的析取范式仍表示同一组非法点。程序从 `minimized.to_dnf().cover` 中逐项读取文字：正文字表示该位固定为 1，负文字表示该位固定为 0，没有出现的变量用 `None` 表示通配位。这里根据文字的唯一标识找到原来的变量下标，使化简前后的坐标始终对应 $(a_3, a_2, a_1, a_0, b_3, b_2, b_1, b_0)$。

立方体到线性约束的转换 对于一个非法立方体，只要当前点有一个固定坐标与它不同，该点就位于立方体之外。因此，对固定为 0 的位置累加 x_j，对固定为 1 的位置累加 $1 - x_j$，并要求总和至少为 1。通配位不参与求和。关键代码为：

```
x = model.addVars(8, vtype=gp.GRB.BINARY)
for cube in cubes:
    model.addConstr(gp.quicksum(
        x[j] if value == 0 else 1 - x[j]
        for j, value in enumerate(cube) if value is not None
    ) >= 1)
```

这样，一条不等式可以排除一个立方体覆盖的多个非法点。Espresso 负责构造较紧凑的非法点覆盖，Gurobi 负责求解转换后的线性约束，两者在程序中依次执行。模型仍只有 8 个原始二进制变量。

[1]PyEDA 的 Espresso 接口及两层逻辑化简说明：https://pyeda.readthedocs.io/en/latest/2llm.html。

实验结果与分析 代码运行结果如图 6所示。



Figure 6: Espresso 逻辑化简后由 Gurobi 枚举得到的结果

本次 Espresso 调用为 DDT 非法点集生成 36 个立方体，为 LAT 非法点集生成 38 个立方体，因此两个模型分别包含 36 条和 38 条不等式。相比逐点排除法的 184 条和 134 条，逻辑化简在保留 8 个变量的同时减少了约束数量。

Gurobi 分别枚举出 72 个差分合法点和 122 个线性合法点，具体输入输出对与前两种方法一致，也与 DDT、LAT 中的非零位置一致。这说明立方体合并及其线性约束转换保持了原有的二进制解集。本实验使用的是 PyEDA 中的 Espresso 化简接口，36 和 38 记录本次得到的覆盖规模，不能据此认定约束数量已经达到全局最少。

4.1.4 凸包法

使用 SageMath 建立完整凸包 实验分为建立凸包和求解两个步骤：`build_hull.py` 在 WSL Arch 中使用 SageMath 计算约束，`convex_hull.py` 在 Windows 的 `uv` 环境中读取约束并调用 Gurobi。

建立凸包时，程序将合法输入输出对编码为八位向量，使用 `QQ` 进行精确有理数计算，并分别提取不等式和等式：

```
points = [bits(a, b) for a in range(SIZE) for b in range(SIZE)
          if table[a][b] != 0]
hull = Polyhedron(vertices=points, base_ring=QQ)

hrep = {
    "inequalities": sorted(integer_row(row)
                           for row in hull.inequalities_list()),
    "equations": sorted(integer_row(row)
                        for row in hull.equations_list()),
}
```

Sage 输出的每行系数按 $[c_0, c_1, \ldots, c_8]$ 排列，第一项为常数，对应 $c_0 + \sum_{j=0}^{7} c_{j+1} x_j \geq 0$，等式使用相同的系数顺序。[2]为便于 JSON 保存和后续整数计算，`integer_row()` 先乘以各系数分母的最小公倍数，再约去整数系数的最大公因数。整个过程使用正数缩放，保持不等式方向。

两组完整 H 表示保存在 `hull_constraints.json` 中，顶层键为 `DDT` 和 `LAT`，各自包含 `inequalities` 与 `equations` 两个系数数组。由于文件位于 Windows 工作目录，WSL 通过`/mnt/d/`访问并生成文件后，Windows 端可以直接读取相同数据。

贪心筛选与 Gurobi 建模 完整 H 表示可以准确描述凸包，但本任务最终只检查二进制点，因此进一步按每条不等式排除非法点的能力进行筛选。程序保留全部等式，将仍满足等式的非法点编号存入 `remaining`，再用 `pool` 记录每条候选不等式能够排除的点集。贪心选择的核心循环为：

```
while remaining:
    row = max(pool, key=lambda item: len(pool[item] & remaining),
              default=None)
    if row is None or not (pool[row] & remaining):
        raise RuntimeError("Incomplete illegal-point coverage")
    chosen.append(row)
    remaining.difference_update(pool.pop(row))
```

其中 `linear_form(row,point)` 计算该行系数在指定点上的值。值小于 0 表示点违反该不等式，因而可以被排除。候选系数按字典序排列，所以并列时选择顺序固定。每次保留的不等式均来自完整凸包，且筛选结束时所有非法点都已被排除，因此约简后的二进制可行点集仍与原合法点集一致。

最后建立 8 个二进制变量，将全部等式以 $h(x) = 0$、筛选后的不等式以 $h(x) \geq 0$ 加入 Gurobi，再按统一解池设置枚举全部可行点。

[2]SageMath 多面体构造与 H 表示说明：https://doc.sagemath.org/html/en/reference/discrete_geometry/sage/geometry/polyhedron/constructor.html。

实验结果与分析 SageMath 计算完整凸包的输出如图 7所示。

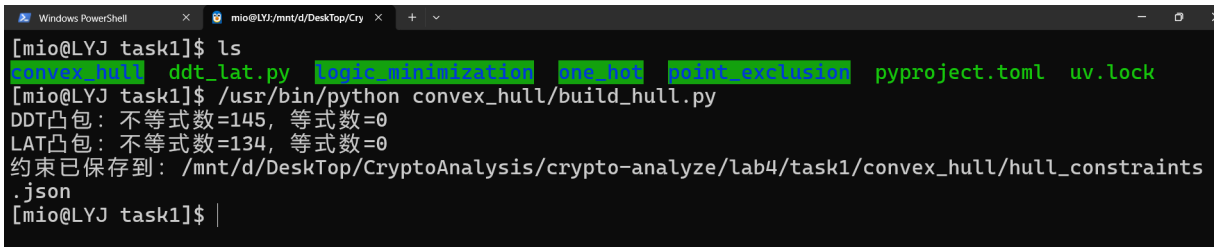


Figure 7: SageMath 建立 DDT、LAT 合法点集凸包并保存 H 表示

DDT 凸包共有 145 条不等式，LAT 凸包共有 134 条不等式，两者等式数均为 0。这与两组合法点集的仿射维数均为 8 相符。整数化后的约束与直接计算表进行核对时，完整 H 表示保留的二进制点分别为 72 个和 122 个，覆盖全部非零位置。

读取 JSON 并进行贪心筛选后，Gurobi 运行结果如图 8所示。



Figure 8: 凸包约束经贪心筛选后的 Gurobi 全解枚举结果

DDT 模型从 145 条不等式中保留 32 条，LAT 模型从 134 条中保留 28 条。由于没有等式，最终加入 Gurobi 的约束数分别就是 32 和 28，两种模型的变量数均为 8。枚举得到的 72 个差分输入输出对与 122 个线性掩码对，与前面三种方法及原始 DDT、LAT 的非零位置完全一致。

进一步对全部 256 个二进制点逐一代入约束，完整 H 表示和筛选后的约束均没有遗漏合法点，也没有保留非法点，说明约束读取、整数化和贪心筛选过程保持了二进制解集。这里的 32 条和 28 条约束用于描述合法二进制点；筛选后的连续可行区域可能大于完整凸包，贪心选择也不保证约束数量全局最少。

4.1.5 四种方法的结果对比

将四种方法的模型规模和枚举结果汇总，如表 1所示。表中的约束数统计显式添加的线性约束，二进制变量的取值范围单独计入变量类型；凸包法使用贪心筛选后实际加入 Gurobi 的约束数

Table 1: 四种方法的模型规模与全解枚举结果

方法	DDT 模型			LAT 模型		
	变量数	约束数	解数	变量数	约束数	解数
one-hot 法	80	9	72	130	9	122
逐点排除法	8	184	72	8	134	122
逻辑化简法	8	36	72	8	38	122
凸包法	8	32	72	8	28	122

四种方法均准确复现 DDT 的 72 个与 LAT 的 122 个非零位置,one-hot 法通过增加选择变量减少线性约束条数，其 9 条约束均为等式；

其余三种方法均使用 8 个变量，其中本次凸包贪心模型的约束数最少

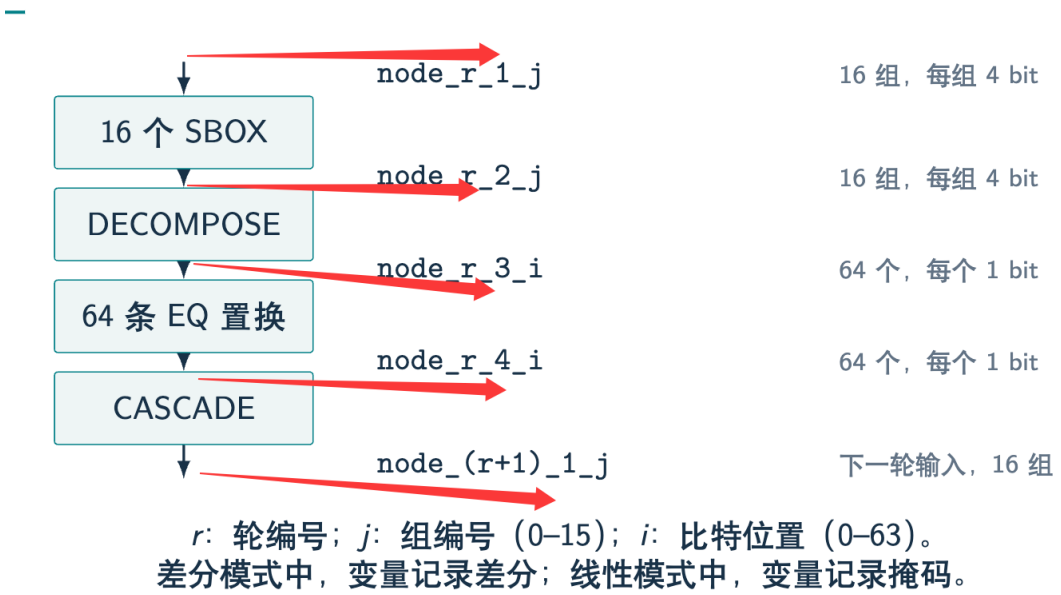
值得注意的是 Espresso 化简和贪心筛选的结果, 都不能保证约束数量全局最少

4.2 任务二：Cipherfour 算法六轮最优差分与线性路线搜索

4.2.1 实验步骤

（1）算法建模与平台配置 本任务利用密码分析学课程实验平台，对 Cipherfour 算法分别进行六轮最优差分路线搜索和最优线性路线搜索

首先我们先理解一下平台这个语法



node 后面跟着三数字，第一个就是轮数, 这个很容易理解

第二个就是层数，这里我们是拆了 4 层，第一层是进 S 盒之前，第二层是出 S 盒之后，要进行 Decompose，那这个 Decompose 就是拆分 bit

因为后面拉线操作是 bit 级别的，然后第三层是 Decompose 之后的，第四层是拉线操作之后的, 然后再经过一个 Cascade, 也就是把前面拆分的合起来，然后就是下一轮了

每一轮本来还要异或轮密钥的，但是异或密钥对差分没有影响，所以说可以忽略

在平台的配置文件中，首先通过 PARAMETERS 指定 S 盒参数，然后使用 SBOX 组件描述每轮的非线性替换操作，通过 DECOMPOSE 和 CASCADE 组件完成状态的拆分与重组，并使用 EQ 组件实现轮间比特置换。

课件里已经给了示例，那这边改一改参数就好了

```
SETTINGS:
START_NODES = [<node_0_1_i,4,i,0,4,1>]
END_NODES =  <node_5_2_i,4,i,0,4,1>]

START_POSITION = 0
END_POSITION = 0
START_VALUE = 0x8
END_VALUE = 0x8

DIFF_UP_BOUND = 128
LINEAR_UP_BOUND = 64
ENDSETTINGS

PARAMETERS:
s = [0x9,0x8,0x0,0x1,0xE,0xB,0xC,0xD,0x3,0x2,0x5,0x7,0xF,0xA,0x6,0x4]
ENDPARAMETERS
```

课件里算法是 16 个 S 盒并行，我们的 Cipherfour 只有 4 个, 然后把 S 盒也修改一下

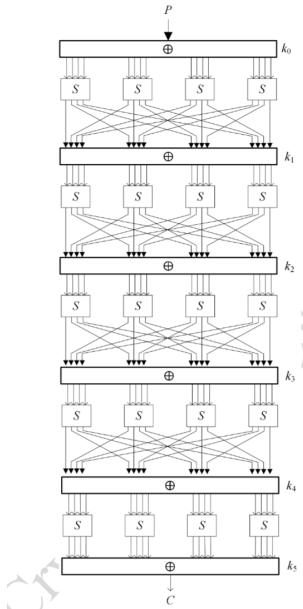


图 A.4: 5轮CipherFour算法加密示意图

Cipherfour 最后一轮是没有 P 置换的，所以说我们最后的状态就取到过完 S 盒就行，也就是第 2 层

所以末尾状态是 node_5_2_i

其中，DIFF_UP_BOUND 用于差分搜索，LINEAR_UP_BOUND 用于线性搜索。两种任务共用同一算法结构，平台根据任务类型自动选择对应的传播约束及优化方式

```
SETTINGS:
START_NODES = [<node_0_1_i,4,i,0,4,1>]
END_NODES = [<node_5_2_i,4,i,0,4,1>]

START_POSITION = 0
END_POSITION = 0
START_VALUE = 0x8
END_VALUE = 0x8

DIFF_UP_BOUND = 128
LINEAR_UP_BOUND = 64
ENDSETTINGS

PARAMETERS:
```

这边 start_value 和 end_value 可以随便填，最后的结果和这个没关系，应该是其他任务使用的

（2）提交任务与调试　完成配置文件编写后，将其上传至课程实验平台，并选择相应的分析任务

差分分析和线性分析只要换一下任务就行，可以用同一个文件

本次实验最终成功完成两项搜索任务，平台申请记录如图 9 所示

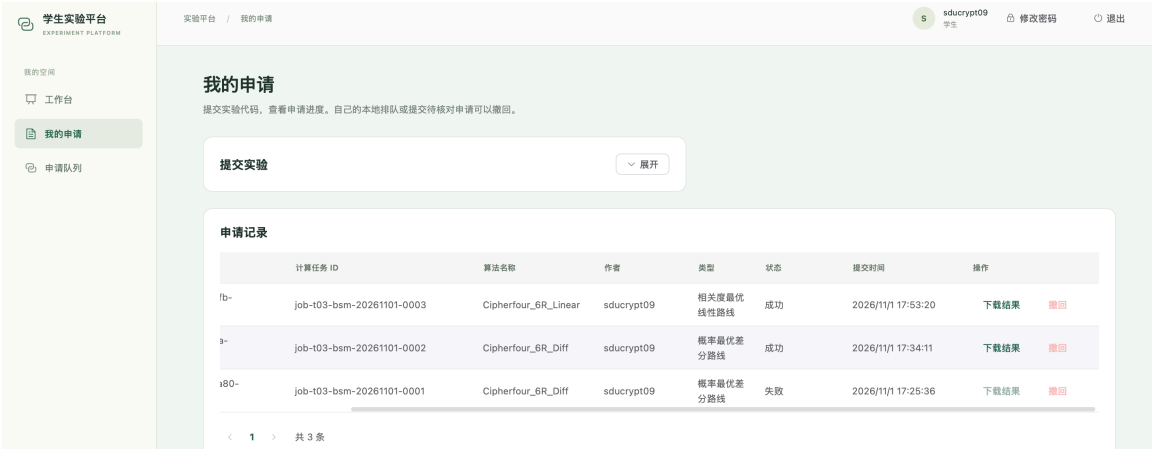
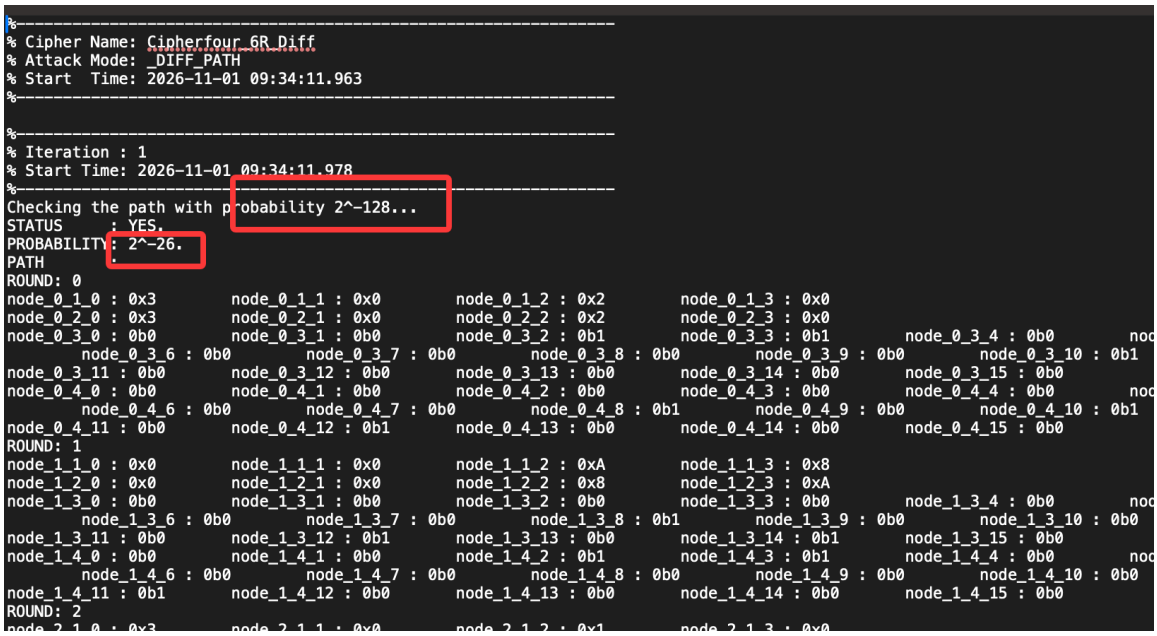


Figure 9: Cipherfour 差分与线性路线搜索任务的提交及运行结果

（3）搜索结果提取与理论验证　平台运行结束后，分别下载差分分析和线性分析对应的 .rpt 与 .cvc 文件。

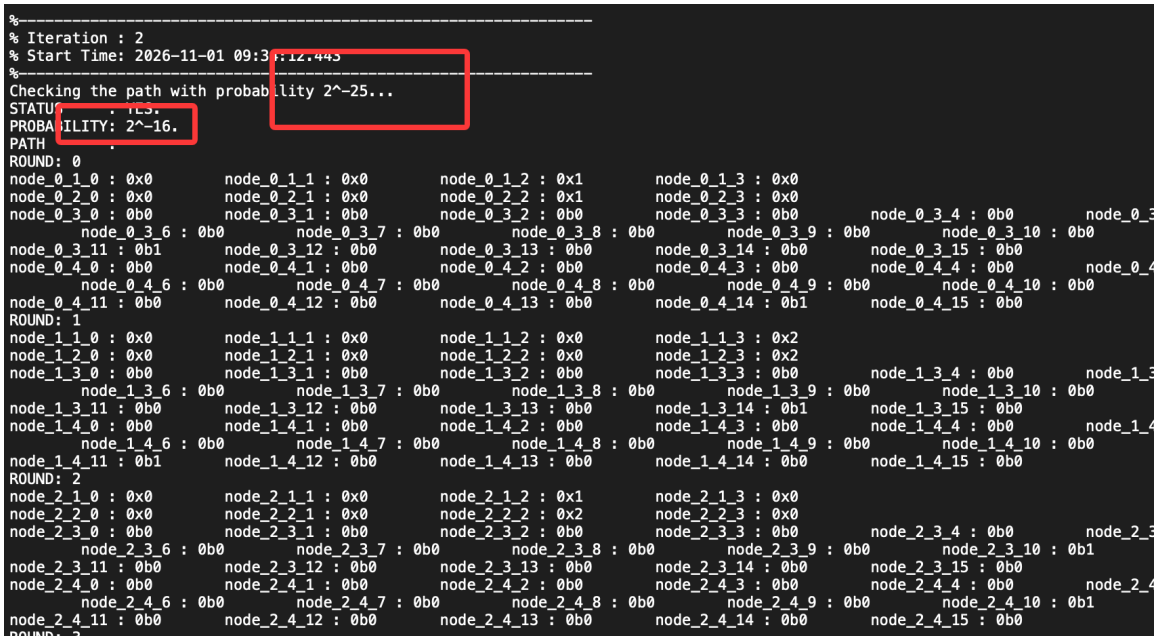
其中，.rpt 文件主要记录迭代过程、搜索状态、概率或相关度权重，以及各轮输入输出差分（或线性掩码）

第一轮迭代就是我们设置的上界 128



然后找到了一条 2^{-26} 概率的路线

下一轮迭代就把 2^{-25} 作为上界继续迭代



.cvc 文件则记录平台生成的约束求解模型。

job-t03-bsm-20261101-0002InV.cvc

```
node_0_1_0:BITVECTOR(4);
node_0_2_0:BITVECTOR(4);
node_0_3_0:BITVECTOR(1);
node_0_3_1:BITVECTOR(1);
node_0_3_2:BITVECTOR(1);
node_0_3_3:BITVECTOR(1);
node_0_1_1:BITVECTOR(4);
node_0_2_1:BITVECTOR(4);
node_0_3_4:BITVECTOR(1);
node_0_3_5:BITVECTOR(1);
node_0_3_6:BITVECTOR(1);
node_0_3_7:BITVECTOR(1);
node_0_1_2:BITVECTOR(4);
node_0_2_2:BITVECTOR(4);
node_0_3_8:BITVECTOR(1);
node_0_3_9:BITVECTOR(1);
node_0_3_10:BITVECTOR(1);
node_0_3_11:BITVECTOR(1);
node_0_1_3:BITVECTOR(4);
node_0_2_3:BITVECTOR(4);
node_0_3_12:BITVECTOR(1);
node_0_3_13:BITVECTOR(1);
node_0_3_14:BITVECTOR(1);
node_0_3_15:BITVECTOR(1);
node_0_4_0:BITVECTOR(1);
```

可以看到就是一些类似我们前面 **gurobi** 的约束，查了一下这是 **STP** 的语法

所以说这个平台应该是用 **STP** 来求解的问题

本实验主要根据 `.rpt` 文件提取最终搜索路线，并通过任务一计算的 **DDT** 与 **LAT** 验证局部 **S** 盒传播的可行性、轮间连接关系及最终权重。

4.2.2 实验结果及分析

（1）六轮最优差分路线搜索 差分分析任务的名称为 `Cipherfour_6R_Diff`。

平台采用逐步收紧概率下界的方式进行搜索。在第 9 次迭代中，平台找到概率为 2^{-6} 的可行差分路线；第 10 次迭代进一步尝试搜索概率不低于 2^{-5} 的路线，但求解结果为不可行。

对应的关键运行结果如下：

```
Iteration : 9
Checking the path with probability 2^-6...
STATUS     : YES.
PROBABILITY: 2^-6.

Iteration : 10
Checking the path with probability 2^-5...
STATUS     : No.
```

由此可知，在当前六轮差分搜索模型下，最优差分路线的权重为

$$W_D = 6$$

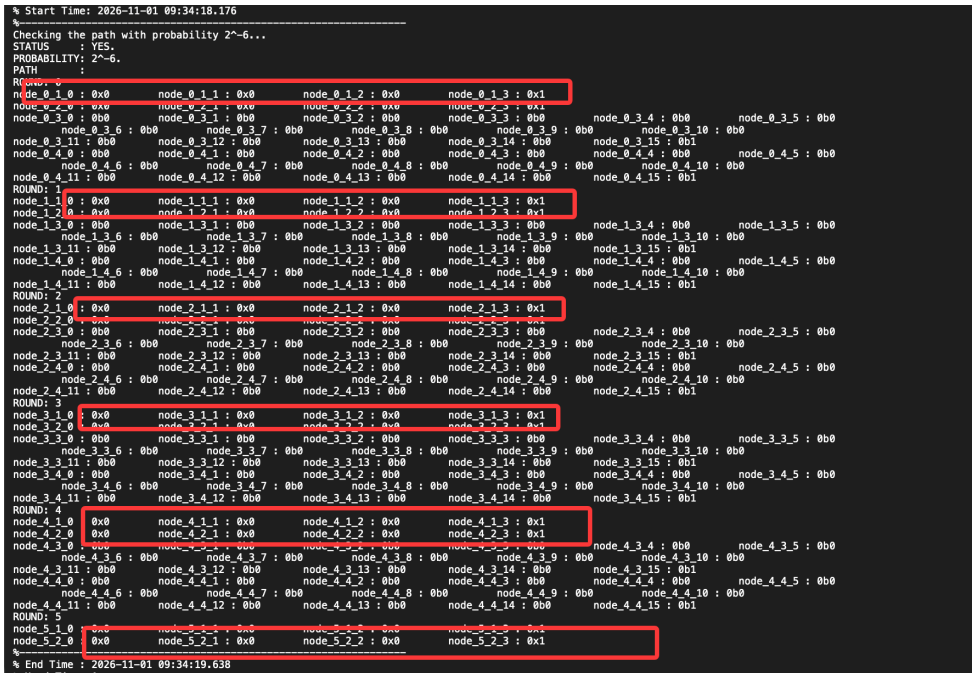
对应的差分路线概率为

$$P_{\text{trail}} = 2^{-6} = \frac{1}{64}$$

从第 9 次迭代的输出中提取六轮差分路线，如表 2 所示。

Table 2: Cipherfour 六轮最优差分路线

轮数	输入差分	输出差分	轮传播概率
1	0x0001	0x0001	2^{-1}
2	0x0001	0x0001	2^{-1}
3	0x0001	0x0001	2^{-1}
4	0x0001	0x0001	2^{-1}
5	0x0001	0x0001	2^{-1}
6	0x0001	0x0001	2^{-1}



可以发现，该路线的六轮输入输出差分均为 0x0001。每轮只有最低四比特对应的 S 盒处于活跃状态，其余三个 S 盒的输入输出差分均为 0。

根据任务一计算得到的 DDT，有

$$\mathrm{DDT}[1][1] = 8, \qquad \mathrm{DDT}[0][0] = 16.$$

因此，活跃 S 盒的差分传播概率为

$$\Pr(1 \to 1) = \frac{\mathrm{DDT}[1][1]}{16} = \frac{8}{16} = \frac{1}{2}.$$

对于其他三个非活跃 S 盒，有

$$\Pr(0 \to 0) = 1.$$

因此，每轮四个 S 盒的联合传播概率为

$$P_r = 1 \times 1 \times 1 \times \frac{1}{2} = \frac{1}{2}.$$

同时，由于该路线的非零差分位位于整个状态的最低位，在所采用的置换中有

$$\pi(15) = 15,$$

因此该差分位经过置换后仍保持在原位置，可以在下一轮继续进行相同的传播。

综合六轮的传播概率，根据差分路线的概率乘积模型，有

$$P_{\text{trail}} = \prod_{r=1}^{6} P_r = \left(\frac{1}{2}\right)^6 = 2^{-6}$$

因此，理论计算结果与平台报告的差分路线概率完全一致。

结合第 10 次迭代中更高概率阈值下无可行路线的结果，可以确认平台在当前差分模型下找到了最优六轮差分路线。

（2）六轮最优线性路线搜索 线性分析任务的名称为 `Cipherfour_6R_Linear`。

与差分分析类似，平台逐步收紧相关度对应的权重上界。

在第 9 次迭代中，平台找到内部权重为 3 的可行线性路线，并报告相关度指标为 2^{-3}；第 10 次迭代继续搜索权重不超过 2 的路线，结果为不可行。

关键运行结果如下：

```
Iteration : 9
Checking the path with correlation of 2^-3...
STATUS      : YES.
LINEAR CORELATION: 2^-3.

Iteration : 10
Checking the path with correlation of 2^-2...
STATUS      : No.
```

因此，平台在其内部整数权重模型下得到的最优值为

$$\widetilde{W}_L = 3$$

第 9 次迭代输出的六轮线性路线如表 3 所示。

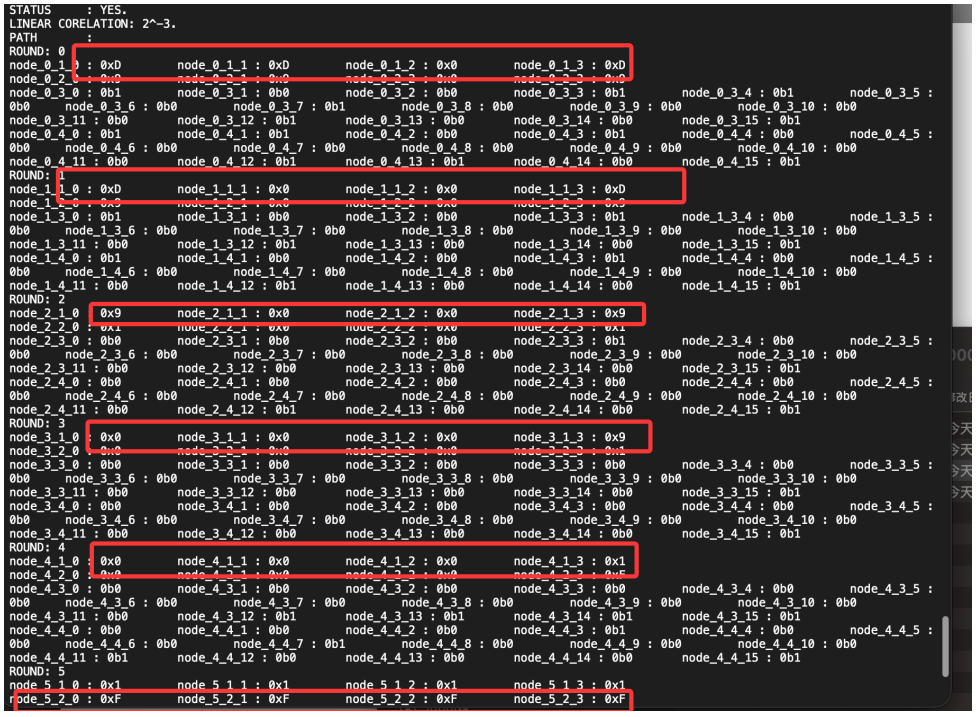


Table 3: Cipherfour 平台搜索得到的六轮线性路线

轮数	输入掩码	输出掩码	精确轮相关度绝对值
1	0xDD0D	0x9909	$(3/4)^3$
2	0xD00D	0x9009	$(3/4)^2$
3	0x9009	0x1001	$(1/2)^2$
4	0x0009	0x0001	$1/2$
5	0x0001	0x000F	$3/4$
6	0x1111	0xFFFF	$(3/4)^4$

为了验证上述路线，首先根据任务一的 LAT 计算各个 S 盒输入输出掩码对的相关度。

对于输入掩码 α 和输出掩码 β，定义 S 盒的 Walsh 相关和为

$$W(\alpha,\beta)=\sum_{x\in\mathbb{F}_2^4}(-1)^{\alpha\cdot x\oplus\beta\cdot S(x)}.$$

其对应的相关度为

$$C(\alpha,\beta)=\frac{W(\alpha,\beta)}{16}.$$

若 LAT 使用偏差计数 $\mathrm{LAT}[\alpha][\beta]=N_0-8$ 的定义，则有

$$W(\alpha,\beta)=2\,\mathrm{LAT}[\alpha][\beta].$$

观察平台给出的线性路线，可以发现其中只涉及以下四种 S 盒局部传播：

Table 4: 线性路线所涉及的 S 盒局部相关度

输入掩码	输出掩码	Walsh 值	相关度
0x0	0x0	16	1
0xD	0x9	12	3/4
0x9	0x1	8	1/2
0x1	0xF	12	3/4

由表 4 可知，上述局部传播的相关度均不为零，因此均属于合法的线性掩码传播。

进一步检查相邻两轮的掩码关系，发现各轮 S 盒输出掩码经过比特置换后，均与下一轮 S 盒的输入掩码一致，说明该路线满足算法的轮间连接约束。

按照线性路线相关度的乘积公式，

$$C_{\text{trail}}=\prod_{r=1}^{6}\prod_{j=0}^{3}C_{r,j},$$

其中 $C_{r,j}$ 表示第 r 轮第 j 个 S 盒的局部相关度。

对该路线中的局部传播进行统计，共有 10 个相关度为 $3/4$ 的传播、3 个相关度为 $1/2$ 的传播，其余 11 个位置的相关度均为 1。

因此，

$$\begin{aligned}|C_{\text{trail}}| &= \left(\frac{3}{4}\right)^{10}\left(\frac{1}{2}\right)^{3} \\ &= \frac{59049}{8388608} \\ &\approx 0.00703919\end{aligned}$$

对应的精确线性权重为

$$\begin{aligned}W_L &= -\log_2 |C_{\text{trail}}| \\ &\approx 7.1504\end{aligned}$$

需要注意的是，精确计算得到的线性权重与平台报告的整数权重 3 并不相同。

为分析这一差异，进一步检查平台生成的 `.cvc` 文件中的 **S** 盒权重编码，发现平台对非零相关度使用的权重符合

$$\widetilde{w}(\alpha,\beta) = \lfloor -\log_2 |C(\alpha,\beta)| \rfloor .$$

例如，对于相关度为 $3/4$ 的局部传播，

$$-\log_2 \frac{3}{4} \approx 0.415,$$

经过向下取整后，平台将其编码为

$$\widetilde{w} = 0.$$

而对于相关度为 $1/2$ 的局部传播，则有

$$\widetilde{w} = -\log_2 \frac{1}{2} = 1.$$

因此，该路线在平台内部的总权重为

$$\begin{aligned}\widetilde{W_L} &= 10 \times 0 + 3 \times 1 + 11 \times 0 \\ &= 3\end{aligned}$$

这一结果与平台报告完全一致。

由此说明，平台报告的 2^{-3} 是其整数化权重模型对应的搜索指标，而根据原始 **LAT** 计算得到的该路线精确相关度绝对值约为 0.00703919。

因此，平台成功找到了整数化权重模型下的最优线性路线，但该结果不能直接作为精确相关度全局最优性的证明。

（3）实验结果总结 本次实验利用课程自动化分析平台，完成了 **Cipherfour** 算法的六轮差分路线和六轮线性路线搜索，并根据任务一计算得到的 **DDT** 和 **LAT** 对搜索结果进行了独立验证

两项搜索结果汇总如表 5 所示。

Table 5: Cipherfour 六轮差分与线性搜索结果汇总

分析类型	平台最优权重	平台报告指标	精确验证结果
差分分析	6	2^{-6}	2^{-6}
线性分析	3	2^{-3}	0.00703919